

WaveWidget & WavePainter

A reusable animated wave background/widget built using **CustomPainter** and **AnimationController**. It draws two animated sine-wave layers moving in opposite directions, each with its own vertical gradient.

✨ Features

- Smooth continuous wave animation
 - Two wave layers moving in opposite directions
 - Fully customizable:
 - Height
 - Wave count (density)
 - Amplitude
 - Wavelength
 - Speed
 - Separate gradients per wave layer
 - Lightweight (no external animation packages)
-

Components Overview

WaveWidget

Controls animation timing and exposes configuration parameters.

WavePainter

Handles all drawing logic using sine functions and gradients.

WaveWidget Parameters

Property	Type	Default	Description
height	double	100	Height of the wave area
gradientColors1	List<Color>	required	Gradient for wave layer 1
gradientColors2	List<Color>	required	Gradient for wave layer 2
lineCount	int	10	Number of wave lines per layer
amplitude	double	20	Wave height (vertical movement)
waveLength	double	200	Horizontal wave length

Property	Type	Default	Description
speed	double	1.0	Animation speed (higher = faster)

WavePainter Parameters

Property	Type	Description
animationValue	double	Value from AnimationController (0–1)
gradientColors1	List<Color>	Gradient colors for wave 1
gradientColors2	List<Color>	Gradient colors for wave 2
lineCount	int	Number of layered strokes
amplitude	double	Wave height
waveLength	double	Horizontal frequency

Usage Example

```
WaveWidget(
  height: 120,
  lineCount: 12,
  amplitude: 24,
  waveLength: 180,
  speed: 1.5,
  gradientColors1: [
    Colors.blue.withOpacity(0.6),
    Colors.purple.withOpacity(0.6),
  ],
  gradientColors2: [
    Colors.cyan.withOpacity(0.5),
    Colors.indigo.withOpacity(0.5),
  ],
)
```

How It Works

- Uses `AnimationController.repeat()` for infinite animation
- Wave motion is created using:

```
y = centerY + amplitude × sin(2π(x / waveLength) ± animation + phase)
```

- Each line has a slight phase offset to create depth
- Stroke width increases gradually for layered effect

Performance Tips

- Reduce `lineCount` on low-end devices
- Keep `amplitude` moderate to avoid overflow
- Use `RepaintBoundary` if embedding in complex UI

Common Customizations

Faster motion

```
speed: 2.0
```

Softer waves

```
amplitude: 12,  
lineCount: 6
```

Dense background effect

```
lineCount: 20
```

Notes

- Gradients are vertical (top → bottom)
 - Waves are centered vertically
 - Widget automatically stretches to full width
-

Full Source Code

wave_widget.dart

```
import 'package:flutter/material.dart';
import 'wavePainter.dart';

class WaveWidget extends StatefulWidget {
  final double height;
  final List<Color> gradientColors1;
  final List<Color> gradientColors2;
  final int lineCount;
  final double amplitude;
  final double waveLength;
  final double speed;

  const WaveWidget({
    super.key,
    this.height = 100,
    required this.gradientColors1,
    required this.gradientColors2,
    this.lineCount = 10,
    this.amplitude = 20,
    this.waveLength = 200,
    this.speed = 1.0,
  });

  @override
  State<WaveWidget> createState() => _WaveWidgetState();
}

class _WaveWidgetState extends State<WaveWidget>
    with SingleTickerProviderStateMixin {
  late final AnimationController _controller;

  @override
  void initState() {
    super.initState();
    final duration = Duration(milliseconds: (4000 ~/ widget.speed));
    _controller = AnimationController(vsync: this, duration: duration)
      ..repeat();
  }

  @override
  void dispose() {
    _controller.dispose();
  }
}
```

```

        super.dispose();
    }

    @override
    Widget build(BuildContext context) {
        return SizedBox(
            height: widget.height,
            width: double.infinity,
            child: AnimatedBuilder(
                animation: _controller,
                builder: (context, child) {
                    return CustomPaint(
                        painter: WavePainter(
                            animationValue: _controller.value,
                            gradientColors1: widget.gradientColors1,
                            gradientColors2: widget.gradientColors2,
                            lineCount: widget.lineCount,
                            amplitude: widget.amplitude,
                            waveLength: widget.waveLength,
                        ),
                    );
                },
            ),
        );
    }
}

```

wavePainter.dart

```

import 'dart:math';
import 'package:flutter/material.dart';

class WavePainter extends CustomPainter {
    final double animationValue;
    final List<Color> gradientColors1;
    final List<Color> gradientColors2;
    final int lineCount;
    final double amplitude;
    final double waveLength;

    WavePainter({
        required this.animationValue,
        required this.gradientColors1,
        required this.gradientColors2,
        this.lineCount = 10,
        this.amplitude = 20,
    }) {}
}

```

```

        this.waveLength = 200,
    });

    @override
    void paint(Canvas canvas, Size size) {
        final paint = Paint()..style = PaintingStyle.stroke;

        // Wave 1
        for (int i = 0; i < lineCount; i++) {
            final path = Path();
            final phase = i * pi / lineCount;

            for (double x = 0; x <= size.width; x++) {
                final y = size.height / 2 +
                    amplitude *
                        sin(2 * pi * (x / waveLength) +
                            animationValue * 2 * pi +
                            phase);
                x == 0 ? path.moveTo(x, y) : path.lineTo(x, y);
            }

            paint
                ..shader = LinearGradient(
                    begin: Alignment.topCenter,
                    end: Alignment.bottomCenter,
                    colors: gradientColors1,
                ).createShader(Rect.fromLTWH(0, 0, size.width, size.height))
                ..strokeWidth = 1 + i / 2;

            canvas.drawPath(path, paint);
        }

        // Wave 2 (opposite direction)
        for (int i = 0; i < lineCount; i++) {
            final path = Path();
            final phase = i * pi / lineCount;

            for (double x = 0; x <= size.width; x++) {
                final y = size.height / 2 +
                    amplitude *
                        sin(2 * pi * (x / waveLength) -
                            animationValue * 2 * pi +
                            phase);
                x == 0 ? path.moveTo(x, y) : path.lineTo(x, y);
            }

            paint
                ..shader = LinearGradient(

```

```

        begin: Alignment.topCenter,
        end: Alignment.bottomCenter,
        colors: gradientColors2,
    ).createShader(Rect.fromLTWH(0, 0, size.width, size.height))
    ..strokeWidth = 1 + i / 2;

    canvas.drawPath(path, paint);
  }
}

@override
bool shouldRepaint(covariant WavePainter oldDelegate) => true;
}

```

If you want: -  Horizontal gradients - Filled waves (water effect) - Direction control - Responsive scaling

Tell me and I'll extend it cleanly.